Камера и отсечение

# Камера и отсечение

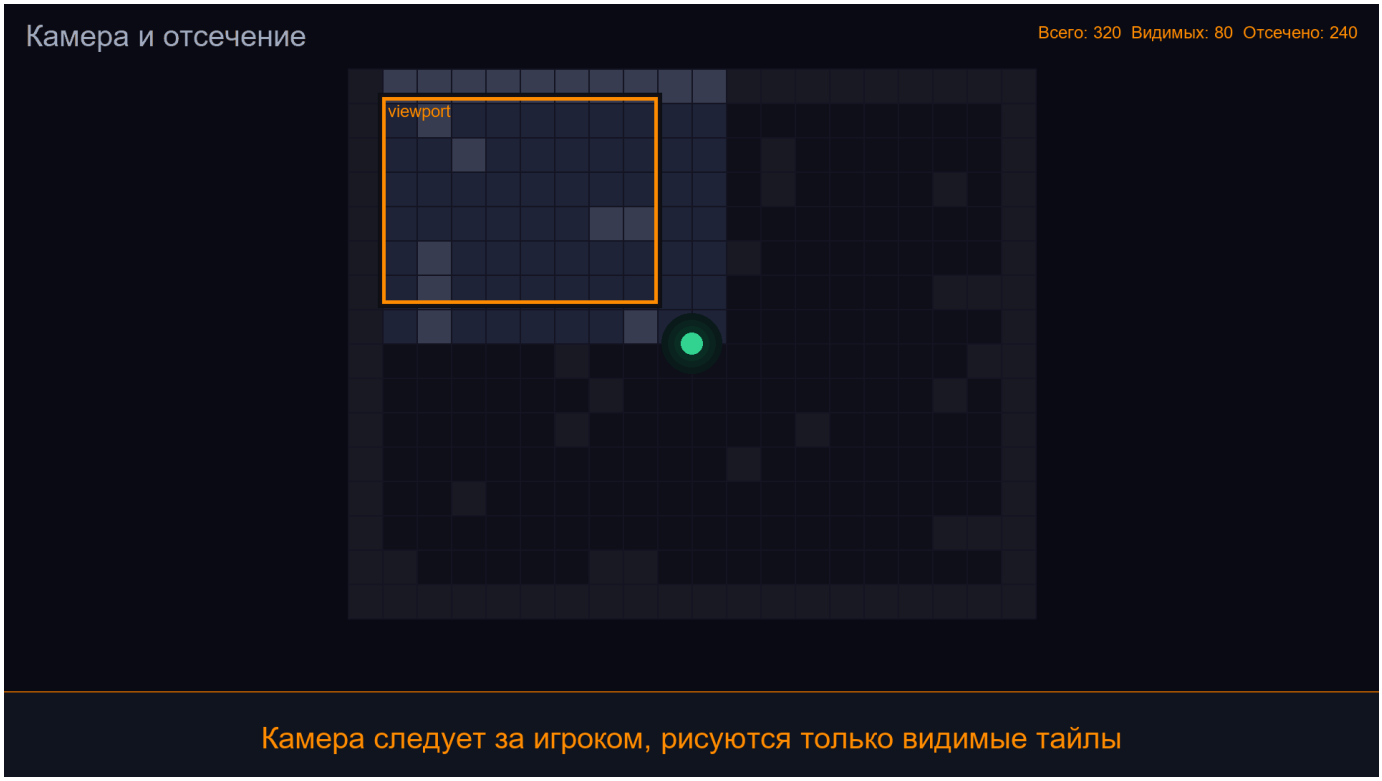
## Проблема

В разделе про уровни мы разбивали ландшафт на тайлы и хранили их в матрице. Допустим, наш уровень - 200x150 тайлов, каждый тайл - 32x32 пикселя. Итого 6400x4800 пикселей. А экран игрока - 1920x1080 (или меньше). Т.е. уровень в несколько раз больше экрана.

Что делать? Отрисовывать весь уровень целиком и сжимать его до размеров экрана? Тогда все будет микроскопическим. Показывать только левый верхний угол? Тогда игрок не увидит остальную часть мира.

Нам нужен механизм, который показывает **часть мира** вокруг игрока и двигается вместе с ним. Как настоящая камера, снимающая актёра на сцене. Актёр идёт - камера следует за ним.

Собственно, этот вопрос уже всплывал ранее. Давайте разберёмся.




## Камера как смещение

Самая простая модель камеры - это точка в мире, определяющая, какую его часть мы видим. У камеры есть координаты `(x, y)` - левый верхний угол видимой области - и размеры (ширина и высота экрана).

Как это работает при отрисовке? Каждый объект в игре хранит свои **мировые координаты** - позицию на уровне. Но на экран мы рисуем не по мировым координатам, а по **экранным**. Экранная координата - это мировая координата минус позиция камеры.

Грубо говоря:

```
screen_x = world_x - camera_x
screen_y = world_y - camera_y
```

Если камера находится в точке `(500, 300)`, а объект в мире стоит в `(600, 400)`, то на экране он будет нарисован в `(100, 100)`. Вот и вся камера. Смещение.

Python C#

```python
class Camera:
    def __init__(self, screen_width, screen_height):
        self.x = 0
        self.y = 0
        self.width = screen_width
        self.height = screen_height

    def follow(self, target_x, target_y):
        # center camera on target
        self.x = target_x - self.width // 2
        self.y = target_y - self.height // 2

    def apply(self, world_x, world_y):
        # convert world coordinates to screen coordinates
        return world_x - self.x, world_y - self.y


# usage in game loop
camera = Camera(800, 600)

while running:
    # ...
    camera.follow(player.x, player.y)

    # draw player on screen
    screen_x, screen_y = camera.apply(player.x, player.y)
    pygame.draw.rect(screen, BLUE, (screen_x, screen_y, 32, 32))

    # draw any object the same way
    for obj in game_objects:


```

```
            sx, sy = camera.apply(obj.x, obj.y)
            pygame.draw.rect(screen, obj.color, (sx, sy, obj.size, obj.size))
```

Метод `follow` ставит камеру так, чтобы цель (игрок) оказался в центре экрана. Метод `apply` переводит мировые координаты в экранные. Всё. Два метода - и у вас работающая камера.

## Плавное следование

Базовая камера мгновенно "прыгает" к игроку. Если игрок резко дёрнулся - камера дёрнулась вместе с ним. Это может выглядеть неприятно, особенно при быстром движении.

Решение: вместо того чтобы мгновенно перемещать камеру к цели, мы **плавно двигаем** её. На каждом кадре камера проходит только часть расстояния до цели. Чем дальше камера от игрока - тем быстрее она движется. Чем ближе - тем медленнее. Это **линейная интерполяция к цели** (lerp toward target) - камера плавно "догоняет" цель.

Помните deltaTime из раздела про игровой цикл? Здесь он нам пригодится - чтобы скорость камеры не зависела от FPS.

Python | C#

```
class Camera:
    def __init__(self, screen_width, screen_height):
        self.x = 0.0
        self.y = 0.0
        self.width = screen_width
        self.height = screen_height

    def smooth_follow(self, target_x, target_y, speed, delta_time):
        # where the camera should be
        target_cx = target_x - self.width / 2
        target_cy = target_y - self.height / 2

        # move a fraction of the distance each frame
        self.x += (target_cx - self.x) * speed * delta_time
        self.y += (target_cy - self.y) * speed * delta_time

    def apply(self, world_x, world_y):
        return world_x - self.x, world_y - self.y


# usage
camera = Camera(800, 600)

while running:
    delta_time = clock.tick(60) / 1000.0  # seconds
```

```
27
28      camera.smooth_follow(player.x, player.y, speed=5.0, delta_time=delta_t:
        # ...
```

Параметр `speed` управляет тем, насколько быстро камера "догоняет" игрока. При `speed=5.0` камера достаточно отзывчива, но при этом плавная. При `speed=2.0` - заметно "ленивая", плывёт за игроком с задержкой. При очень большом значении (50+) плавность исчезает и камера ведёт себя как мгновенная. Формула - приближение: при стабильных 30+ FPS она работает отлично, но при сильных просадках FPS или огромном `speed` камера может проскочить цель.

Попробуйте разные значения speed и посмотрите, что вам больше нравится.

## Ограничения камеры

Если игрок стоит у левого края уровня, камера может "выехать" за пределы мира, и на экране появится пустота. Это выглядит плохо. Поэтому нужно ограничить положение камеры - она не должна показывать то, чего нет.

Для этого **зажимаем** (clamp) координаты камеры в допустимых пределах:

Python C#

```python
def clamp(self, level_width, level_height):
    # don't go past the left/top edge
    self.x = max(0, self.x)
    self.y = max(0, self.y)
    # don't go past the right/bottom edge
    self.x = min(self.x, level_width - self.width)
    self.y = min(self.y, level_height - self.height)


# usage: call after follow/smooth_follow
camera.smooth_follow(player.x, player.y, 5.0, delta_time)
camera.clamp(level_pixel_width, level_pixel_height)
```

Где `level_width` и `level_height` - размеры уровня **в пикселях** (количество тайлов * размер тайла). Теперь камера не покажет пустоту за краями уровня.

Если ваш уровень меньше экрана (например, одна маленькая комната), то clamp может работать некорректно - `level_width - screen_width` станет отрицательным. В

этом случае можно просто отцентрировать камеру на уровне.

## Отсечение (Culling)

Ок. Камера работает, мы видим нужную часть мира. Но вот вопрос: если уровень - 200x150 тайлов, а на экране помещается 25x19, зачем нам на каждом кадре обрабатывать и рисовать все 30 000 тайлов?

Правильно, незачем. Нужно рисовать только то, что **попадает в область видимости** камеры. Это и называется **отсечение** (culling) - мы "отсекаем" невидимые объекты. Тема упоминалась в разделе про коллизии, здесь мы реализуем её конкретно для камеры.

### Отсечение тайлов

Для тайловой карты всё просто: зная позицию камеры и размер тайла, мы можем вычислить, какие строки и столбцы матрицы видны на экране, и рисовать только их.

Python C#

```python
def get_visible_range(self, tile_size, cols, rows):
    start_col = max(0, int(self.x // tile_size))
    start_row = max(0, int(self.y // tile_size))
    end_col = min(cols, int((self.x + self.width) // tile_size) + 2)
    end_row = min(rows, int((self.y + self.height) // tile_size) + 2)
    return start_col, end_col, start_row, end_row


# usage in draw
tile_size = 32
cols, rows = 200, 150  # level size in tiles
c1, c2, r1, r2 = camera.get_visible_range(tile_size, cols, rows)

for row in range(r1, r2):
    for col in range(c1, c2):
        tile = level[row][col]
        world_x = col * tile_size
        world_y = row * tile_size
        screen_x, screen_y = camera.apply(world_x, world_y)
        # draw tile at (screen_x, screen_y)
```

Обратите внимание на `+ 2` в `end_col` и `end_row`. Мы добавляем запас, чтобы тайлы, частично попадающие в кадр, тоже были отрисованы. Без этого у краёв экрана могут появляться "щели".

Вместо 30 000 тайлов мы рисуем ~500. Разница ощутимая, особенно на больших уровнях.

### Отсечение объектов

С тайлами всё просто - они лежат в сетке, и мы можем точно вычислить видимый диапазон. А что с произвольными объектами - врагами, предметами, снарядами?

Для них проверяем, попадает ли объект в видимую область камеры. По сути, это та же проверка пересечения прямоугольников (AABB), только один из прямоугольников - камера.

Python C#

```python
def is_visible(self, obj_x, obj_y, obj_width, obj_height):
    return (obj_x + obj_width > self.x
            and obj_x < self.x + self.width
            and obj_y + obj_height > self.y
            and obj_y < self.y + self.height)


# usage
for enemy in enemies:
    if camera.is_visible(enemy.x, enemy.y, enemy.width, enemy.height):
        sx, sy = camera.apply(enemy.x, enemy.y)
        # draw enemy
```

Отсечение влияет только на отрисовку. Игровая логика (перемещение врагов, физика, таймеры) обычно обрабатывается для всех объектов, независимо от того, видит их камера или нет. Иначе враги будут "замораживаться", стоит им выйти за край экрана. Хотя это может быть и багом, и фичей...

## Масштабирование (Zoom)

Камера может не только двигаться, но и менять масштаб. Приближение - видим меньше мира, но крупнее. Отдаление - видим больше, но мельче.

Реализация: помимо смещения, мы добавляем множитель масштаба. При отрисовке экранная координата считается так:

```
screen_x = (world_x - camera_x) * zoom
screen_y = (world_y - camera_y) * zoom
```

При `zoom = 1.0` - стандартный вид. При `zoom = 2.0` - всё в два раза крупнее. При `zoom = 0.5` - в два раза мельче (видим в четыре раза больше).

Где это может пригодиться? Например, миникарта - это та же камера, но с маленьким `zoom`, рисующая в угол экрана. Или эффект приближения при прицеливании.

| Что | Где подробнее |
| --- | --- |
| Тайлы, граф уровня, откуда берётся матрица | Уровни и поиск путей |
| Пространственные структуры для ускорения отсечения | Пространственные структуры |
| deltaTime для плавного следования | Основной цикл |
| Коллизии с миром и отсечение | Коллизии |
| ИИ врагов: видимость и off-screen поведение | ИИ врагов |

Powered by Wiki.js